

```
char str[20];  
printf("Enter the string:");  
scanf("%s",str);  
printf("The string reversed is:%s",strrev(str));  
getch();  
}
```

OUTPUT : Enter the string :subbareddy

The string reversed is : ydderabbus

UNIT – III

FUNCTIONS:



User-Defined Functions

Definition: A function is a block of code/group of statements/self contained block of statements/basic building blocks in a program that performs a particular task. It is also known as *procedure* or *subroutine* or **module**, in other programming languages.

To perform any task, we can create function. A function can be called many times. It provides *modularity* and code *reusability*.

Advantage of functions

1) Code Reusability

By creating functions in C, you can call it many times. So we don't need to write the same code again and again.

2) Code optimization

It makes the code optimized we don't need to write much code.

3) Easily to debug the program.

Example: Suppose, you have to check 3 numbers (781, 883 and 531) whether it is prime number or not. Without using function, you need to write the prime number logic 3 times. So, there is repetition of code.

But if you use functions, you need to write the logic only once and you can reuse it several times.

Types of Functions

There are two types of functions in C programming:

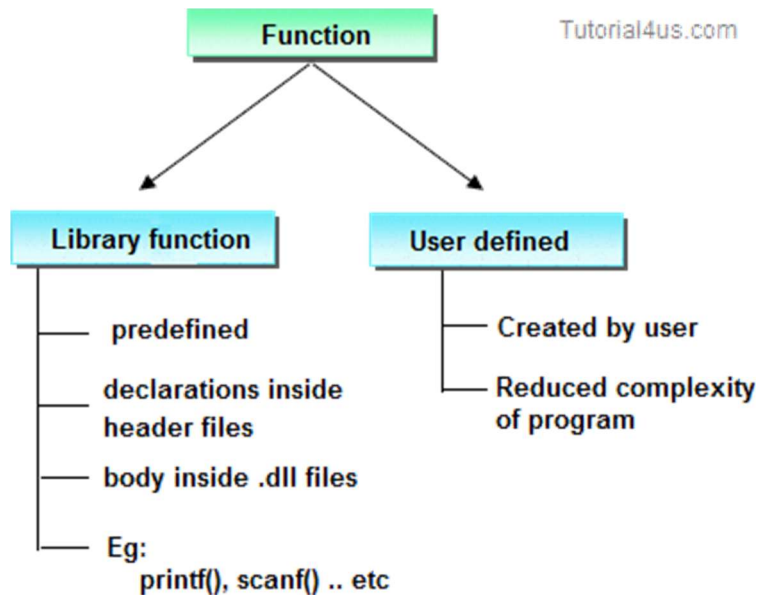
1. **Library Functions:** are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc. You just need to include appropriate header files to use these functions. **These are already declared and defined in C libraries. oints to be Remembered**

System defined functions are declared in header files

System defined functions are implemented in .dll files. (DLL stands for Dynamic Link Library).

To use system defined functions the respective header file must be included.

2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code. Depending upon the complexity and requirement of the program, you can create as many user-defined functions as you want.



ELEMENTS OF USER-DEFINED FUNCTIONS :

In order to write an efficient user defined function, the programmer must be familiar with the following three elements.

1 : Function Declaration. (Function Prototype).

2 : Function Call.

3 : Function Definition

Function Declaration. (Function Prototype).

A function declaration is the process of telling the compiler about a function name.

Syntax

```
return_type function_name(parameter/argument);
```

```
return_type function-name();
```

Ex : `int add(int a,int b);`

`int add();`

Note: At the time of function declaration function must be terminated with ;.

Calling a function/function call

When we call any function control goes to function body and execute entire code.

Syntax : function-name();
function-name(parameter/argument);
return value/ variable = function-name(parameter/argument);

Ex : add(); // function without parameter/argument
 add(a,b); // function with parameter/argument
 c=fun(a,b); // function with parameter/argument and return values

Defining a function.

Defining of function is nothing but give body of function that means write logic inside function body.

Syntax

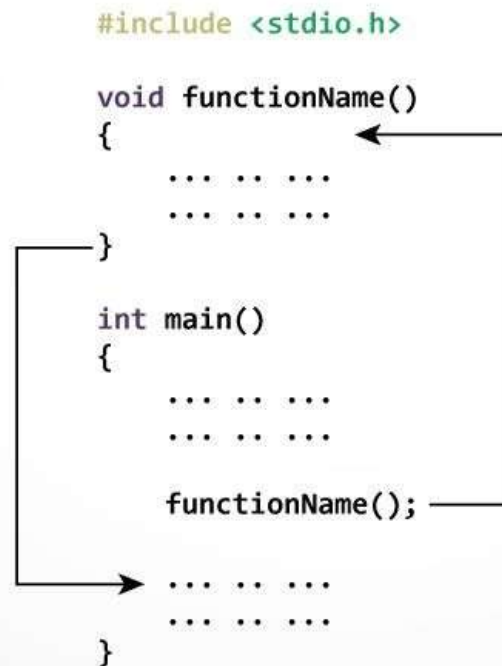
return_type function-name(parameter list) // **function header.**

```
{  
    declaration of variables;  
    body of function;                // Function body  
    return statement; (expression or value) // optional  
}
```

Eg:	int add(int x, int y)	int add(int x, int y)
	{	{
	int z; (or)	return (x + y);
	z = x + y;	}

```
    return z;  
}
```

How function works in C programming?



The execution of a C program begins from the main() function.

When the compiler encounters `functionName();` inside the main function, control of the program jumps to

`void functionName()`

And, the compiler starts executing the codes inside the user-defined function.

The control of the program jumps to statement next to `functionName();` once all the codes inside the function definition are executed.

Example:

```
#include<stdio.h>
```

```

#include<conio.h>

void sum(); // declaring a function

clrset();

int a=10,b=20, c;

void sum() // defining function
{
c=a+b;
printf("Sum: %d", c);
}

void main()
{
sum(); // calling function
}

```

Output

Sum:30

Example:

```

#include <stdio.h>

int addNumbers(int a, int b);    // function prototype

int main()
{
    int n1,n2,sum;

    printf("Enters two numbers: ");

    scanf("%d %d",&n1,&n2);

    sum = addNumbers(n1, n2);    // function call
}

```

```

printf("sum = %d",sum);

return 0;

}

int addNumbers(int a,int b)    // function definition

{

    int result;

    result = a+b;

    return result;            // return statement

}

```

How to pass arguments to a function?

```

#include <stdio.h>

int addNumbers(int a, int b);

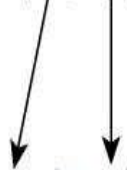
int main()
{
    ... .. ...

    sum = addNumbers(n1, n2);

    ... .. ...
}

int addNumbers(int a, int b)
{
    ... .. ...
    ... .. ...
}

```



Return Statement

Syntax of return statement

Syntax : `return;` **// does not return any value**

or

`return(exp);` **// the specified exp value to calling function.**

For example,

`return a;`

`return (a+b);`

The return statement terminates the execution of a function and returns a value to the calling function. The program control is transferred to the calling function after return statement.

In the above example, the value of variable result is returned to the variable sum in the main() function.

Return statement of a Function

```
#include <stdio.h>

int addNumbers(int a, int b);

int main()
{
    ... ..
    sum = addNumbers(n1, n2);
    ... ..
}

int addNumbers(int a, int b)
{
    ... ..
    return result;
}
```

sum = result

PARAMETERS :

parameters provides the data communication between the calling function and called function.

They are two types of parametes

1 : Actual parameters.

2 : Formal parameters.

1 : Actual Parameters : These are the parameters transferred from the calling function (main program) to the called function (function).

2 : Formal Parameters : These are the parameters transferred into the calling function (main program) from the called function(function).

- The parameters specified in calling function are said to be Actual Parameters.
- The parameters declared in called function are said to be Formal Parameters.
- The value of actual parameters is always copied into formal parameters.

```
Ex :    main()
        {
            fun1( a , b ); //Calling function
        }
        fun1( x, y )    //called function
        {
            . . . . .
        }
```

Where

a, b are the **Actual Parameters**

x, y are the **Formal Parameters**

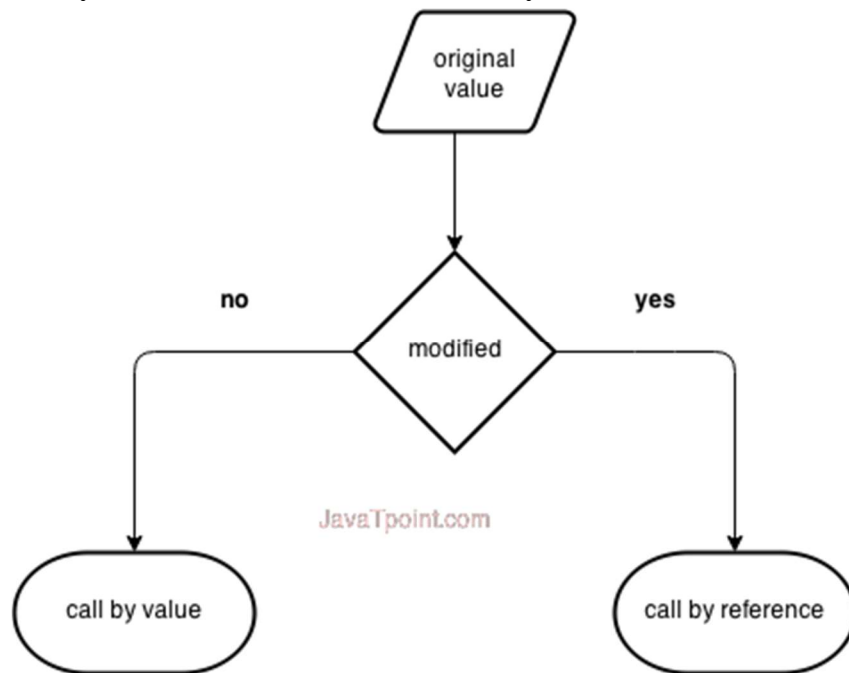
Difference between Actual Parameters and Formal Parameters

Actual Parameters	Formal Parameters
1 : Actual parameters are used in calling function when a function is invoked.	1 : Formal parameters are used in the function header of a called function.

Ex : c=add(a,b); Here a,b are actual parameters.	Ex : int add(int m,int n); Here m,n are called formal parameters.
2 : Actual parameters can be constants, variables or expression. Ex : c=add(a,b) //variable c=add(a+5,b); //expression. c=add(10,20); //constants.	2 : Formal parameters should be only variable. Expression and constants are not allowed. Ex : int add(int m,n); //CORRECT int add(int m+n,int n) //WRONG int add(int m,10); //WRONG
3 : Actual parameters sends values to the formal parameters. Ex : c=add(4,5);	3 : Formal parameters receive values from the actual parameters. Ex : int add(int m,int n); Here m will have the value 4 and n will have the value 5.
4 : Address of actual parameters can be sent to formal parameters	4 : if formal parameters contains address, they should be declared as pointers.

PASSING PARAMETERS TO FUNCTIONS : There are two ways to pass value or data to function in C language: *call by value* and *call by reference*. Original value is **not modified in**

call by value but it is modified in call by reference.



The called function receives the information from the calling function through the parameters. The variables used while invoking the calling function are called actual parameters and the variables used in the function header of the called function are called formal parameters.

C provides two mechanisms to pass parameters to a function.

1 : Pass by value (OR) Call by value.

2 : Pass by reference (OR) Call by Reference.

1 : Pass by value (OR) Call by value :

In call by value, value being passed to the function is locally stored by the function parameter in stack memory location. If you change the value of function parameter, it is changed for the current function only. It will not change the value of variable inside the caller method such as `main()`.

Or

When a function is called with actual parameters, the values of actual parameters are copied into formal parameters. If the values of the formal parameters change in the function, the values of the actual parameters are not changed. This way of passing parameters is called pass by value or call by value.

Ex :

```
#include<stdio.h>

#include<conio.h>

void swap(int ,int );

void main()
{
    int i,j;

    printf("Enter i and j values:");

    scanf("%d%d",&i,&j);

    printf("Before swapping:%d%d\n",i,j);

    swap(i,j);

    printf("After swapping:%d%d\n",i,j);

    getch();
}

void swap(int a,int b)
{
    int temp;

    temp=a;

    a=b;

    b=temp;
}
```

Output

Enter i and j values: 10 20

Before swapping: 10 20

After swapping: 10 20

2 : Pass by reference (OR) Call by Reference : In pass by reference, a function is called with addresses of actual parameters. In the function header, the formal parameters receive the addresses of actual parameters. Now the formal parameters do not contain values, instead they contain addresses. Any variable if it contains an address, it is called a pointer variable. Using pointer variables, the values of the actual parameters can be changed. This way of passing parameters is called call by reference or pass by reference.

```
Ex :          #include<stdio.h>

              #include<conio.h>

              void swap(int *,int *);

              void main()
              {
                  int i,j;

                  printf("Enter i and j values:");

                  scanf("%d%d",&i,&j);

                  printf("Before swapping:%d%d\n",i,j);

                  swap(&i ,&j);

                  printf("After swapping:%d%d\n",i,j);

              }

              void swap(int *a,int *b)
              {

                  int temp;

                  temp=*a;

                  *a=*b;

                  *b=temp;
```

}

Output

Enter i and j values: 10 20

Before swapping:10 20

After swapping: 20 10

Difference between Call by value and Call by reference

Call by value	Call by Reference
1 : When a function is called the values of variables are passed	1 : When a function is called the address of variables are passed.
2 : Change of formal parameters in the function will not affect the actual parameters in the calling function.	2 : The actual parameters are changed since the formal parameters indirectly manipulate the actual parameters.
3 : Execution is slower since all the values have to be copied into formal parameters.	3 : Execution is faster since only address are copied.

1 : Functions with no Parameters and no Return Values.

2 : Functions with no Parameters and Return Values.

3 : Functions with Parameters and no Return Values.

4 : Functions with Parameters and Return Values.

1 : Functions with no Parameters and no Return Values :

1 : In this category, there is no data transfer between the calling function and called function.

2 : But there is flow of control from calling function to the called function.

3 : When no parameters are there , the function cannot receive any value from the calling function.

4: When the function does not return a value, the calling function cannot receive any value from the called function.

Ex `#include<stdio.h>`


```

#include<conio.h>

void sum();

void main()
{
    sum();

    getch();
}

void sum()
{
    int a,b,c;

    printf("enter the values of a and b");

    scanf("%d%d",&a,&b);

    c=a+b;

    printf("sum=%d",c);
}

```

2 : Functions with no Parameters and Return Values.

- 1 : In this category, there is no data transfer between the calling function and called function.
- 2 : But there is data transfer from called function to the calling function.
- 3 : When no parameters are there , the function cannot receive any values from the calling function.
- 4: When the function returns a value, the calling function receives one value from the called function.

Ex :

```

#include<stdio.h>

#include<conio.h>

int sum();

```

```

void main()
{
    int c;

    clrscr();

    c=sum();

    printf("sum=%d",c);

    getch();
}

int sum()
{
    int a,b,c;

    printf("enter the values of a and b");

    scanf("%d%d",&a,&b);

    c=a+b;

    return c;
}

```

3 : Functions with Parameters and no Return Values.

1 : In this category, there is data transfer from the calling function to the called function using parameters.

2 : But there is no data transfer from called function to the calling function.

3 : When parameters are there, the function can receive any values from the calling function.

4: When the function does not return a value, the calling function cannot receive any value from the called function.

Ex :

```

#include<stdio.h>

#include<conio.h>

```

```

void sum(int a,int b);

void main()
{
    int m,n;

    clrscr();

    printf("Enter m and n values:");

    scanf("%d%d",&m,&n);

    sum(m,n);

    getch();
}

void sum(int a,int b)
{
    int c;

    c=a+b;

    printf("sum=%d",c);
}

```

4 : Functions with Parameters and Return Values.

1 : In this category, there is data transfer from the calling function to the called function using parameters.

2 : But there is no data transfer from called function to the calling function.

3 : When parameters are there, the function can receive any values from the calling function.

4: When the function returns a value, the calling function receive a value from the called function.

Ex :

```

#include<stdio.h>

#include<conio.h>

int sum(int a,int b);

```

```

void main()
{
    int m,n,c;

    clrscr();

    printf("Enter m and n values");

    scanf("%d%d",&m,&n);

    c=sum(m,n);

    printf("sum=%d",c);

    getch();
}

int sum(int a,int b)
{
    int c;

    c=a+b;

    return c;
}

```

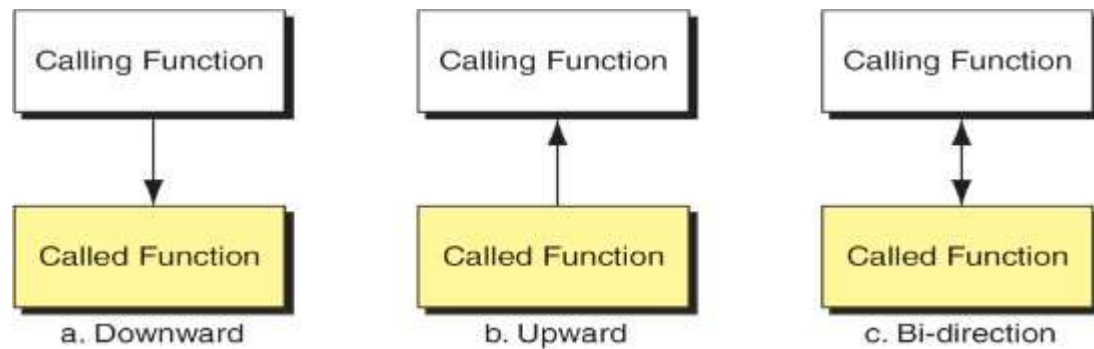
Inter-Function Communication

When a function gets executed in the program, the execution control is transferred from calling function to called function and executes function definition, and finally comes back to the calling function. In this process, both calling and called functions have to communicate each other to exchange information. The process of exchanging information between calling and called functions is called as inter function communication.

In C, the inter function communication is classified as follows...

- Downward Communication
- Upward Communication

- Bi-directional Communication



Downward Communication

In this type of inter function communication, **the data is transferred from calling function to called function** but not from called function to calling function. The functions with parameters and without return value are considered under downward communication. In the case of downward communication, the execution control jumps from calling function to called function along with parameters and executes the function definition, and finally comes back to the calling function without any return value. For example consider the following program...

Example:

```
#include <stdio.h>

#include<conio.h>

void main(){

    int num1, num2 ;

    void addition(int, int) ; // function declaration

    clrscr() ;

    num1 = 10 ;

    num2 = 20 ;

    printf("\nBefore swap: num1 = %d, num2 = %d", num1, num2) ;

    addition(num1, num2) ; // calling function

    getch() ;
```

```

}

void addition(int a, int b) // called function

{
    printf("SUM = %d", a+b) ;
}

```

Output

SUM=30

Upward Communication

In this type of inter function communication, the **data is transferred from called function to calling function** but not from calling function to called function. The functions without parameters and with return value are considered under upward communication. In the case of upward communication, the execution control jumps from calling function to called function without parameters and executes the function definition, and finally comes back to the calling function along with a return value. For example consider the following program...

Exmample:

```

#include <stdio.h>

#include<conio.h>

void main(){
    int result ;

    int addition() ; // function declaration

    clrscr() ;

    result = addition() ; // calling function

    printf("SUM = %d", result) ;

    getch() ;
}

int addition() // called function

```

```

{
    int num1, num2 ;

    num1 = 10;

    num2 = 20;

    return (num1+num2) ;
}

```

Output

SUM=30

Bi - Directional Communication

In this type of inter function communication, the data is transferred from calling function to called function and also from called function to calling function. The functions with parameters and with return value are considered under bi-directional communication. In the case of bi- directional communication, the execution control jumps from calling function to called function along with parameters and executes the function definition, and finally comes back to the calling function along with a return value. For example consider the following program...

Example:

```

#include <stdio.h>

#include<conio.h>

void main(){

    int num1, num2, result ;

    int addition(int, int) ; // function declaration

    clrscr() ;

    num1 = 10 ;

    num2 = 20 ;

    result = addition(num1, num2) ; // calling function

    printf("SUM = %d", result) ;
}

```

```

    getch() ;

}

int addition(int a, int b) // called function

{

    return (a+b) ;

}

```

Output

SUM=30

Standard Functions

The standard functions are built-in functions. In C programming language, the standard functions are declared in header files and defined in .dll files. In simple words, the standard functions can be defined as "the ready made functions defined by the system to make coding more easy". The standard functions are also called as library functions or pre-defined functions.

In C when we use standard functions, we must include the respective header file using #include statement. For example, the function printf() is defined in header file stdio.h (Standard Input Output header file). When we use printf() in our program, we must include stdio.h header file using #include<stdio.h> statement.

C Programming Language provides the following header files with standard functions.

Header File	Purpose	Example Functions
stdio.h	Provides functions to perform standard I/O operations	printf(), scanf()
conio.h	Provides functions to perform console I/O operations	clrscr(), getch()
math.h	Provides functions to perform mathematical operations	sqrt(), pow()
string.h	Provides functions to handle string data values	strlen(), strcpy()
stdlib.h	Provides functions to perform general functions	calloc(), malloc()

time.h	Provides functions to perform operations on time and date	time(), localtime()
ctype.h	Provides functions to perform - testing and mapping of character data values	isalpha(), islower()
setjmp.h	Provides functions that are used in function calls	setjump(), longjump()
signal.h	Provides functions to handle signals during program execution	signal(), raise()
assert.h	Provides Macro that is used to verify assumptions made by the program	assert()
locale.h	Defines the location specific settings such as date formats and currency symbols	setlocale()
stdarg.h	Used to get the arguments in a function if the arguments are not specified by the function	va_start(), va_end(), va_arg()
errno.h	Provides macros to handle the system calls	Error, errno
float.h	Provides constants related to floating point data values	
limits.h	Defines the maximum and minimum values of various variable types like char, int and long	
stddef.h	Defines various variable types	
graphics.h	Provides functions to draw graphics.	circle(), rectangle()

STANDARD „C“ LIBRARY FUNCTIONS

1 : stdio.h

2 : stdlib.h

3 : string.h

4 : math.h

5 : ctype.h

6 : time.h

1 : STANDARD I/O LIBRARY FUNCTIONS <STDIO.H>

Functions	Data Type	Purpose
printf()	int	Send data items to the standard output device.
scanf()	int	Enter data items from the standard input device.
gets(s)	char	Enter string s from the standard input device.
getc(f)	int	Enter a string character from file f.
getchar()	int	Enter a single character from the standard input device.
putc(c,f)	int	Send a single character to file f.
puts(s)	int	Send string s to the standard output device.
putchar(c)	int	Send a single character to the standard output device.
fgetc(f)	int	Enter a single character from file f.
fgets(s,I,f)	char	Enter string s, containing I characters, from file f.
fprintf(f)	int	Send data items to file f.
fscanf(f)	int	Enter data items from file f.
fputc(c,f)	int	Send a single character to file f.
fputs(s,f)	int	Send string s to file f.
fread(s,il,i2,f)	int	Enter i2 data items, each of size il bytes, from file f.
fclose(f)	int	Close file f, return 0 if file is successfully closed.

2 : STANDARD LIBRARY FUNCTIONS <STDLIB.H>

Functions	Data Type	Purpose
abs(i)	int	Return the absolute value of i.
atof(s)	double	Convert string s to a double-precision quantity.
calloc(u1,u2)	void*	Allocate memory for an array having u1 elements, each of length u2 bytes. Return a pointer to the beginning of the allocated space.
exit(u)	void	Close all files and buffers, and terminate the program.
free(p)	void	Free a block of allocated memory whose beginning is indicated by p.
malloc(u)	void*	Allocate u bytes of memory.
rand()	int	Return a random positive integer.
realloc(p,u)	void*	Allocate u bytes of new memory to the pointer variable p, return a pointer to the beginning of the new memory space.
system(s)	int	Pass command string s to the operating system.
srand(u)	void	Initialize the random number generator.

3 : STRING LIBRARY FUNCTIONS <STRING.H>

Functions	Data Type	Purpose
strlen()		Finds length of string
strlwr()		Converts a string to lowercase
strupr()		Converts a string to uppercase
strcat()		Appends one string at the end of another
strcpy()		Copies a string into another
strcmp()		Compares two strings
strrev()		Reverses string

4 : MATH LIBRARY FUNCTIONS <MATH.H>

Functions	DataType	Purpose
acos(d)	double	Return the arc cosine of d.
atan(d)	double	Return the arc tangent of d.
asin(d)	double	Return the arc sine of d.
ceil(d)	double	Return a value rounded up to the next higher integer.
cos(d)	double	Return the cosine of d.
cosh(d)	double	Return the hyperbolic cosine of d.
exp(d)	double	Raise e to the power d.
fabs(d)	double	Return the absolute value of d.
floor(d)	double	Return a value rounded down to the next lower integer.
labs(l)	long int	Return the absolute value of l.
log(d)	double	Return the natural logarithm of d.
pow(d1,d2)	double	Return d1 raised to the d2 power.
sin(d)	double	Return the sine of d.
sqrt(d)	double	Return square root of d.
tan(d)	double	Return the tangent of d.

5 : CHARACTER LIBRARY FUNCTIONS <CTYPE.H>

Functions	DataType	Purpose
-----------	----------	---------

isalnum(c)	Int	Determine if argument is alphanumeric. Return nonzero value if true, 0 otherwise.
isalpha(c)	Int	Determine if argument is alphabetic. Return nonzero value if true, 0 otherwise.
isascii(c)	Int	Determine if argument is an ASCII character,. Return nonzero value if true, 0 otherwise.
isdigit(c)	Int	Determine if argument is a decimal digit. Return nonzero value if true, 0 otherwise.
isgraph(c)	Int	Determine if argument is a graphic printing ASCII Character. Return nonzero value if true, 0 otherwise.
islower(c)	Int	Determine if argument is lowercase. Return nonzero value if true, 0 otherwise.
isprint(c)	Int	Determine if argument is a printing ASCII character. Return nonzero value if true, 0 otherwise.
isspace(c)	Int	Determine if argument is a whitespace character. Return nonzero value if true, 0 otherwise.
isupper(c)	Int	Determine if argument is uppercase. Return nonzero value if true, 0 otherwise.
toascii(c)	Int	Convert value of argument to ASCII
tolower(c)	Int	Convert letter to lowercase
toupper(c)	Int	Convert letter to uppercase.

6 : TIME LIBRARY FUNCTIONS <TIME.H>

Functions	Data Type	Purpose
difftime(11,12)	double	Return the time difference 11-12, where 11 and 12 represent elapsed time beyond a designated base time.
time(p)	long int	Return the number of seconds elapsed beyond a designated base time.

Storage Classes

In C language, each variable has a storage class which is used to define scope and life time of a variable.

Storage: Any variable declared in a program can be stored either in memory or registers. Registers are small amount of storage in CPU. The data stored in registers has fast access compared to data stored in memory.

Storage class of a variable gives information about the location of the variable in which it is stored, initial value of the variable, if storage class is not specified; scope of the variable; life of the variable.

There are four storage classes in C programming.

1 : Automatic Storage class.

2 : Register Storage class.

3 : Static Storage class.

4 : External Storage class.

1: Automatic Storage class : To define a variable as automatic storage class, the keyword „auto“ is used. By defining a variable as automatic storage class, it is stored in the memory. The default value of the variable will be garbage value. Scope of the variable is within the block where it is defined and the life of the variable is until the control remains within the block.

Syntax : auto data_type variable_name;

```
auto int a,b;
```

Example:

```
void main()
{
    int detail;

    or

    auto int detail;  //Both are same
}
```

The variables a and b are declared as integer type and auto. The **keyword auto is not mandatory**. Because the **default storage class in C is auto**.

Note: A variable declared inside a function without any storage class specification, is **by default an automatic variable**. Automatic variables can also be called local variables because they are local to a function.

Ex :	<pre>void function1(); void function2(); void main() { int x=100; function2(); printf("%d",x); }</pre>	OUTPUT
	<pre>void function1() { int x=10; printf("%d",x); }</pre>	10
	<pre>void function2() { int x=0; function1(); printf("%d",x); }</pre>	0
		100

2: Register Storage class : To define a variable as register storage class, the **keyword „register“ is used**. If CPU cannot store the variables in CPU registers, then the variables are

assumed as auto and stored in the memory. **When a variable is declared as register, it is stored in the CPU registers.** The **default value of the variable will be garbage value.** Scope of the variable within the block where it is defined and the life of the variables is until the control remains within the block.

Register variable has faster access than normal variable. Frequently used variables are kept in register. Only few variables can be placed inside register.

NOTE : We can't get the address of register variable

Syntax : register data_type variable_name;

Ex: register int i;

Ex :	OUTPUT
void demo();	
void main()	20
{	20
demo();	20
demo();	
demo();	
}	
void demo()	
{	
register int i=20;	
printf(“%d\n”,i);	
i++;	
}	

3 : Static Storage class : When a variable is declared as static, it is stored in the memory. The **default value of the variable will be zero.** Scope of the variable is within the block where it is defined and the life of the variable persists between different function calls. To define a variable as static storage class, **the keyword „static“ is used.** A static variable can be initialized only once, it cannot be reinitialized.

Syntax : static data_type variable_name;

Ex: static int i;

Ex :		OUTPUT
	void demo();	
	void main()	20
	{	21
	demo();	22
	demo();	
	demo();	
	}	
	void demo()	
	{	
	static int i=20;	
	printf(“%d”,i);	
	i++;	
	}	

4 : External Storage class : When a **variable is declared as extern**, it is stored in the memory. The **default value is initialized to zero**. The scope of the variable is global and the life of the variable is until the program execution comes to an end. To define a variable as external storage class, the **keyword „extern“** is used. An extern variable is also called as a global variable. **Global** variables remain available throughout the entire program. One important thing to remember about global variable is that their values can be changed by any function in the program.

Syntax : extern data_type variable_name;
extern int i;

Ex:

```
int number;
```

```
void main()
```

```
{
```

```
    number=10;
```

```
}
```

```
fun1()
```

```
{
```

```
    number=20;
```

```
}
```

```
fun2()
```

```
{
```

```
    number=30;
```

```
}
```

Here the global variable **number** is available to all three functions.

Ex : void fun1();

 void fun2();

 int e=20;

 void main()

 {

 fun1();

 fun2();

 }

 void fun1()

```

    {
        extern int e;

        printf("e number is :%d",e);
    }

    void fun2()
    {
        printf("e number is :%d",e);
    }

```

extern keyword

The extern keyword is used before a variable to inform the compiler that this variable is declared somewhere else. The extern declaration does not allocate storage for variables.

Problem when extern is not used

```

main()
{
    a = 10;    //Error:cannot find variable a

    printf("%d",a);
}

```

Example Using extern in same file

```

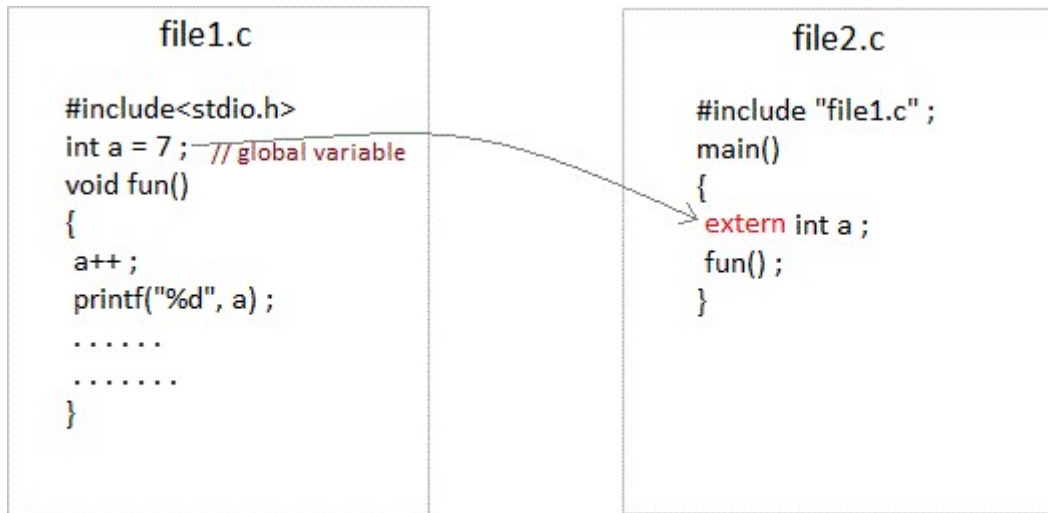
main()
{
    extern int x; //Tells compiler that it is defined somewhere else

    x = 10;

    printf("%d",x);
}

```

```
int x; //Global variable x
```



global variable from one file can be used in other using **extern** keyword.

Storage Classes	Storage Place	Default Value	Scope	Life-time
auto	RAM	Garbage Value	Local	Within function
extern	RAM	Zero	Global	Till the end of main program, May be declared anywhere in the program
static	RAM	Zero	Local	Till the end of main program, Retains value between multiple functions call
register	Register	Garbage Value	Local	Within function

Recursion

When *function is called within the same function*, it is known as recursion in C. The function which calls the same function, is known as recursive function.

A function that calls itself, and doesn't perform any task after function call, is known as tail recursion. In tail recursion, we generally call the same function with return statement.

Features :

- There should be at least one if statement used to terminate recursion.
- It does not contain any looping statements.

Advantages :

- It is easy to use.
- It represents compact programming structures.

Disadvantages :

- It is slower than that of looping statements because each time function is called.

Note: while using recursion, programmers need to be careful to define an exit condition from the function, otherwise it will go into an infinite loop. Recursive functions are very useful to solve many mathematical problems, such as calculating the factorial of a number, generating Fibonacci series, etc.

Example of recursion.

```
recursionfunction(){  
  
    recursionfunction();//calling self function  
  
}
```

How does recursion work?

```
void recurse()
{
    ... ..
    recurse();
    ... ..
}

int main()
{
    ... ..
    recurse();
    ... ..
}
```

Example of tail recursion in C

// print factorial number using tail recursion

```
#include<stdio.h>

#include<conio.h>

int factorial (int n)
{
    if ( n < 0)
        return -1; /*Wrong value*/

    if (n == 0)
        return 1; /*Terminating condition*/

    return (n * factorial (n -1));
}

void main(){
```

```
int fact=0;
clrscr();
fact=factorial(5);
printf("\n factorial of 5 is %d",fact);
getch(); } Outputfactorial of 5 is 120
```

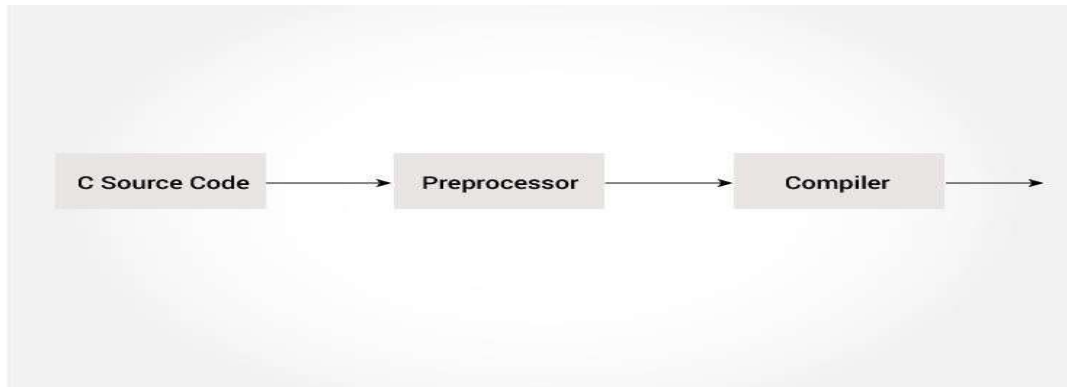
return 5 * factorial(4) = 120
└─ return 4 * factorial(3) = 24
 └─ return 3 * factorial(2) = 6
 └─ return 2 * factorial(1) = 2
 └─ return 1 * factorial(0) = 1

javaTpoint.com

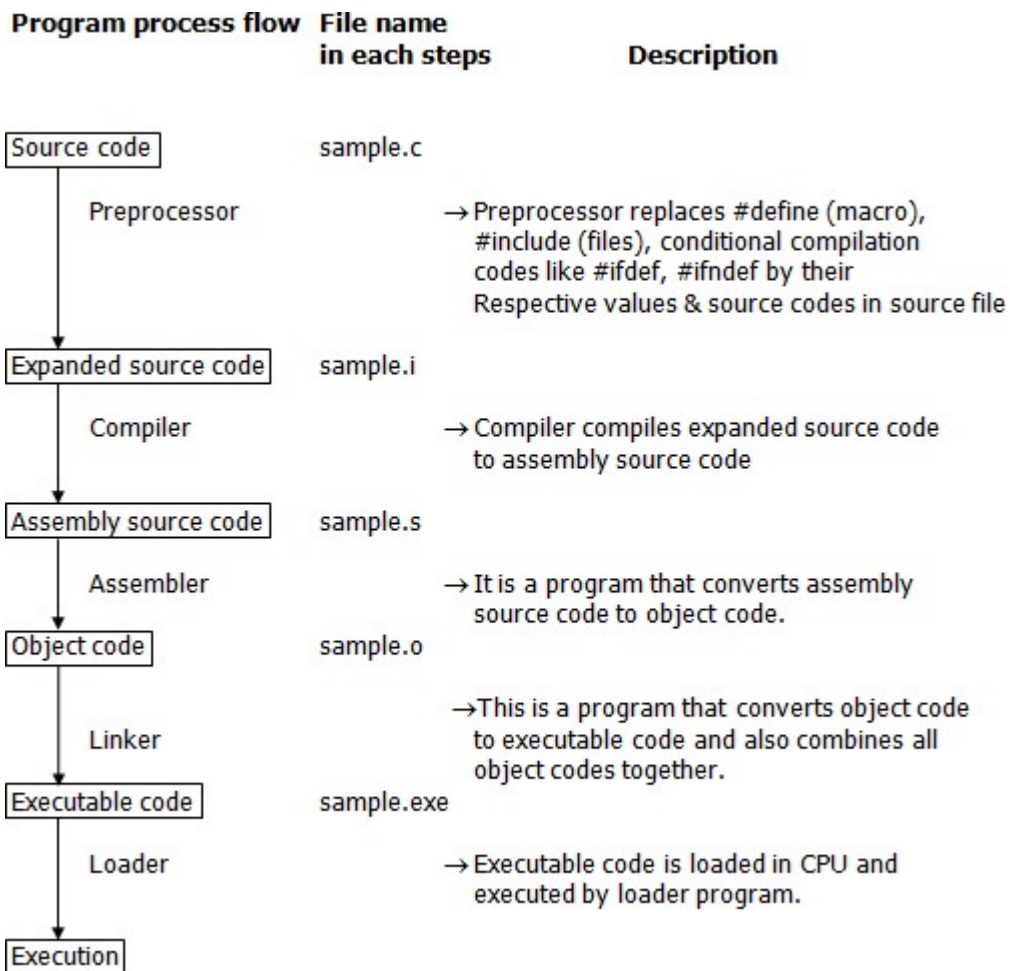
$1 * 2 * 3 * 4 * 5 = 120$

Fig: Recursion

Preprocessor Commands



A program in C language involves into different processes. Below diagram will help you to understand all the processes that a C program comes across.



The **C Preprocessor** is not a part of the compiler, but is a separate step in the compilation process. In simple terms, a C Preprocessor is just a text substitution tool and it instructs the

compiler to do required pre-processing before the actual compilation. All preprocessor commands begin with a hash symbol (#).

list of preprocessor directives.

- #include
- #define
- #undef
- #ifdef
- #ifndef
- #if
- #else
- #elif
- #endif
- #error
- #pragma

C Macros

A macro is a segment of code which is replaced by the value of macro. Macro is defined by #define directive. There are two types of macros:

1. Object-like Macros
2. Function-like Macros

Object-like Macros

The object-like macro is an identifier that is replaced by value. It is widely used to represent numeric constants. For example:

```
#define PI 3.14
```

Here, PI is the macro name which will be replaced by the value 3.14.

Function-like Macros

The function-like macro looks like function call. For example:

```
#define MIN(a,b) ((a)<(b)?(a):(b))
```

Here, MIN is the macro name.

Predefined Macros

There are some predefined macros which are readily for use in C programming.

No.	Macro	Description
1	<code>_DATE_</code>	represents current date in "MMM DD YYYY" format.
2	<code>_TIME_</code>	represents current time in "HH:MM:SS" format.
3	<code>_FILE_</code>	represents current file name.
4	<code>_LINE_</code>	represents current line number.
5	<code>_STDC_</code>	It is defined as 1 when compiler complies with the ANSI standard.

```
#include <stdio.h>
```

```
main() {
```

```
    printf("File :%s\n", _FILE__);
```

```
    printf("Date :%s\n", _DATE__);
```

```
    printf("Time :%s\n", _TIME__);
```

```
    printf("Line :%d\n", _LINE__);
```

```
    printf("ANSI :%d\n", _STDC__);
```

```
}
```

Output

File :test.c

Date :Jun 2 2012

Time :03:36:24

Line :8

ANSI :1

C #include

The **#include** preprocessor directive is used to paste code of given file into current file. It is used include system-defined and user-defined header files. If included file is not found, compiler renders error.

By the use of **#include** directive, we provide information to the preprocessor where to look for the header files. There are two variants to use **#include** directive.

1. **#include <filename>**
2. **#include "filename"**

The **#include <filename>** tells the compiler to look for the directory where system header files are held. In UNIX, it is `\usr\include` directory.

The **#include "filename"** tells the compiler to look in the current directory from where program is running.

#include directive example

Let's see a simple example of **#include** directive. In this program, we are including `stdio.h` file because `printf()` function is defined in this file.

1. **#include <stdio.h>**
2. **main() {**
3. **printf("Hello C");**
4. **}**

Output:

Hello C

#include notes:

Note 1: In **#include** directive, comments are not recognized. So in case of **#include <a/b>**, `a/b` is treated as filename.

Note 2: In **#include** directive, backslash is considered as normal text not escape sequence. So in case of **#include <a\nb>**, `a\nb` is treated as filename.

Note 3: You can use only comment after filename otherwise it will give error.

C #define

The #define preprocessor directive is used to define constant or micro substitution. It can use any basic data type.

Syntax:

#define token value

Example of #define to define a constant.

```
#include <stdio.h>

#define PI 3.14

main() {
    printf("%f",PI);
}
```

Output:

3.140000

Example of #define to create a macro.

```
#include <stdio.h>

#define MIN(a,b) ((a)<(b)?(a):(b))

void main() {
    printf("Minimum between 10 and 20 is: %d\n", MIN(10,20));
}
```

Output:

Minimum between 10 and 20 is: 10

C #undef

The #undef preprocessor directive is used to undefine the constant or macro defined by #define.

Syntax:

```
#undef token
```

Simple example to define and undefine a constant.

```
#include <stdio.h>
```

```
#define PI 3.14
```

```
#undef PI
```

```
main() {
```

```
    printf("%f",PI);
```

```
}
```

Output:

Compile Time Error: 'PI' undeclared

The #undef directive is used to define the preprocessor constant to a limited scope so that you can declare constant again.

Let's see an example where we are defining and undefining number variable. But before being undefined, it was used by square variable.

```
#include <stdio.h>
```

```
#define number 15
```

```
int square=number*number;
```

```
#undef number
```

```
main() {
```

```
    printf("%d",square);
```

```
}
```

Output:

225

C #ifdef

The #ifdef preprocessor directive checks if macro is defined by #define. If yes, it executes the code otherwise #else code is executed, if present.

Syntax:

```
#ifdef MACRO
```

```
//code
```

```
#endif
```

Syntax with #else:

```
#ifdef MACRO
```

```
//successful code
```

```
#else
```

```
//else code
```

```
#endif
```

C #ifdef example

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define NOINPUT
```

```
void main() {
```

```
int a=0;
```

```
#ifdef NOINPUT
```

```
a=2;
```

```
#else
```

```
printf("Enter a:");
```

```
scanf("%d", &a);
```

```
#endif
```

```
printf("Value of a: %d\n", a);  
getch();  
}
```

Output:

Value of a: 2

if you don't define NOINPUT, it will ask user to enter a number.

```
#include <stdio.h>  
#include <conio.h>  
  
void main() {  
    int a=0;  
    #ifndef NOINPUT  
        a=2;  
    #else  
        printf("Enter a:");  
        scanf("%d", &a);  
    #endif  
    printf("Value of a: %d\n", a);  
    getch();  
}
```

Output:

Enter a:5

Value of a: 5

C #ifndef

The #ifndef preprocessor directive checks if macro is not defined by #define. If yes, it executes the code otherwise #else code is executed, if present.

Syntax:

```
#ifndef MACRO  
  
//code  
  
#endif
```

Syntax with #else:

```
#ifndef MACRO  
  
//successful code  
  
#else  
  
//else code  
  
#endif
```

C #ifndef example

simple example to use #ifndef preprocessor directive.

```
#include <stdio.h>  
  
#include <conio.h>  
  
#define INPUT  
  
void main() {  
  
    int a=0;  
  
    #ifndef INPUT  
  
        a=2;  
  
    #else  
  
        printf("Enter a:");  
  
        scanf("%d", &a);
```



```
#endif  
  
printf("Value of a: %d\n", a);  
  
getch();  
  
}
```

Output:

Enter a:5

Value of a: 5

if you don't define INPUT, it will execute the code of #ifndef.

```
#include <stdio.h>  
  
#include <conio.h>  
  
void main() {  
  
    int a=0;  
  
    #ifndef INPUT  
  
        a=2;  
  
    #else  
  
        printf("Enter a:");  
  
        scanf("%d", &a);  
  
    #endif  
  
    printf("Value of a: %d\n", a);  
  
    getch();  
  
}
```

Output:

Value of a: 2

C #if

The #if preprocessor directive evaluates the expression or condition. If condition is true, it executes the code otherwise #elseif or #else or #endif code is executed.

Syntax:

```
#if expression
```

```
//code
```

```
#endif
```

Syntax with #else:

```
#if expression
```

```
//if code
```

```
#else
```

```
//else code
```

```
#endif
```

Syntax with #elif and #else:

```
#if expression
```

```
//if code
```

```
#elif expression
```

```
//elif code
```

```
#else
```

```
//else code
```

```
#endif
```

C #if example

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define NUMBER 0
```

```
void main() {  
    #if (NUMBER==0)  
        printf("Value of Number is: %d",NUMBER);  
    #endif  
    getch();  
}
```

Output:

Value of Number is: 0

Another example to understand the #if directive clearly.

```
#include <stdio.h>  
#include <conio.h>  
#define NUMBER 1  
  
void main() {  
    clrscr();  
    #if (NUMBER==0)  
        printf("1 Value of Number is: %d",NUMBER);  
    #endif  
    #if (NUMBER==1)  
        printf("2 Value of Number is: %d",NUMBER);  
    #endif  
    getch();  
}
```

Output:

2 Value of Number is: 1

C #else

The #else preprocessor directive evaluates the expression or condition if condition of #if is false. It can be used with #if, #elif, #ifdef and #ifndef directives.

Syntax:

```
#if expression  
  
//if code  
  
#else  
  
//else code  
  
#endif
```

Syntax with #elif:

```
#if expression  
  
//if code  
  
#elif expression  
  
//elif code  
  
#else  
  
//else code  
  
#endif
```

C #else example

```
#include <stdio.h>  
  
#include <conio.h>  
  
#define NUMBER 1  
  
void main() {  
  
#if NUMBER==0  
  
printf("Value of Number is: %d",NUMBER);  
  
#else
```

```
print("Value of Number is non-zero");  
  
#endif  
  
getch();  
  
}
```

Output:

Value of Number is non-zero

C #error

The #error preprocessor directive indicates error. The compiler gives fatal error if #error directive is found and skips further compilation process.

C #error example

```
#include<stdio.h>  
  
#ifndef __MATH_H  
  
#error First include then compile  
  
#else  
  
void main(){  
  
    float a;  
  
    a=sqrt(7);  
  
    printf("%f",a);  
  
}  
  
#endif
```

Output:

Compile Time Error: First include then compile

if you include math.h, it does not gives error.

```
#include<stdio.h>  
  
#include<math.h>
```

```
#ifndef __MATH_H
#error First include then compile
#else

void main(){

    float a;

    a=sqrt(7);

    printf("%f",a);

}

#endif
```

Output:

2.645751

C #pragma

The #pragma preprocessor directive is used to provide additional information to the compiler. The #pragma directive is used by the compiler to offer machine or operating-system feature.

Syntax:

#pragma token

Different compilers can provide different usage of #pragma directive.

The turbo C++ compiler supports following #pragma directives.

```
#pragma argsused
#pragma exit
#pragma hdrfile
#pragma hdrstop
#pragma inline
#pragma option
#pragma saveregs
```

```
#pragma startup
```

```
#pragma warn
```

Example to use #pragma preprocessor directive.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void func() ;
```

```
#pragma startup func
```

```
#pragma exit func
```

```
void main(){
```

```
printf("\nI am in main");
```

```
getch();
```

```
}
```

```
void func(){
```

```
printf("\nI am in func");
```

```
getch();
```

```
}
```

Output:

I am in func

I am in main

I am in func

KEY POINTS TO REMEMBER:

1. Source program is converted into executable code through different processes like precompilation, compilation, assembling and linking.
2. Local variables uses stack memory.
3. Dynamic memory allocation functions use the heap memory.

preprocessor	Syntax/Description
Macro	Syntax: #define This macro defines constant value and can be any of the basic data types.
Header file inclusion	Syntax: #include <file_name> The source code of the file “file_name” is included in the main program at the specified place.
Conditional compilation	Syntax: #ifdef, #endif, #if, #else, #ifndef Set of commands are included or excluded in source program before compilation with respect to the condition.
Other directives	Syntax: #undef, #pragma #undef is used to undefine a defined macro variable. #Pragma is used to call a function before and after main function in a C program.

DIFFERENCE BETWEEN STACK & HEAP MEMORY IN C LANGUAGE?

Stack	Heap
Stack is a memory region where “local variables”, “return addresses of function calls” and “arguments to functions” are hold while C program is executed.	Heap is a memory region which is used by dynamic memory allocation functions at run time.

CPU's current state is saved in stack memory	Linked list is an example which uses heap memory.
--	---

DIFFERENCE BETWEEN COMPILERS VS INTERPRETERS IN C LANGUAGE?

Compilers	Interpreters
Compiler reads the entire source code of the program and converts it into binary code. This process is called compilation. Binary code is also referred as machine code, executable, and object code.	Interpreter reads the program source code one line at a time and executing that line. This process is called interpretation.
Program speed is fast.	Program speed is slow.
One time execution. Example: C, C++	Interpretation occurs at every line of the program. Example: BASIC

The following section lists down all the important preprocessor directives –

Directive	Description
#define	Substitutes a preprocessor macro.
#include	Inserts a particular header from another file.

#undef	Undefines a preprocessor macro.
#ifdef	Returns true if this macro is defined.
#ifndef	Returns true if this macro is not defined.
#if	Tests if a compile time condition is true.
#else	The alternative for #if.
#elif	#else and #if in one statement.
#endif	Ends preprocessor conditional.
#error	Prints error message on stderr.
#pragma	Issues special commands to the compiler, using a standardized method.

There are three types of preprocessor commands.

- 1 : macro substitution.
- 2 : file inclusion.
- 3 : conditional compilation directives.

1: Macro Substitution : They are two types of macro substitution.

- 1 : Macro substitution without arguments.
- 2 : Macro substitution with arguments.

1 : Macro substitution without arguments : It is a process to substitute the constant or value in the place of an identifier. It is possible to achieve this with the help of directive or macro definition statement **#define**.

Syntax : #define identifier constant or expression

Ex : #define PI 3.142
 #define MAX_MARKS 100
 #define MIN_MARKS 35

Ex :

```
#include <stdio.h>

#define height 100
#define number 3.14
#define letter 'A'
#define letter_sequence "ABC"
#define backslash_char '\\'

void main()
{
    printf("value of height  : %d \n", height );
    printf("value of number : %f \n", number );
    printf("value of letter : %c \n", letter );
    printf("value of letter_sequence : %s \n", letter_sequence);
    printf("value of backslash_char : %c \n", backslash_char);
}
```

OUTPUT:

value of height : 100

value of number : 3.140000

value of letter : A

value of letter_sequence : ABC



value of backslash_char : ?

Ex : **Example of Macro substitution**

```
#include<stdio.h>

#define PI 3.142

void main()

{

    int r;

    float area;

    printf("Enter the radius of circle");

    scanf("%d",&r);

    area=PI*r*r;

    printf("the area of a circle is%d",area);

}
```

Example of Macro definition with expressions

```
#define A (20*10)

#define B (200-100)

void main()

{

    int div;

    div=A/B;

    printf("the division of two numbers%d",div);

}
```

Example of Macro definition with conditional expression

```
#define IFCONDITION if(a>b)

#define PRINT printf("the value of a is the greatest no")

void main()
{
    int a=100,b=50;

    IFCONDITION

    PRINT;
}
```

Macro Substitution with Arguments :

Syntax : #define identifier(var1,var2,va3,...varn)string

Where **identifier** is the name of macro function with the list of macro formal parameters **var1,var2,var3,...varn** like the formal parameters in a function definition.

Ex : #define PROD(x) (x*x)

```
void main()
{
    int a,mul;

    printf("enter the value of a");

    scanf("%d",&a);

    mul=PROD(a);

    printf("The multiplication of two numbers%d",mul);
}
```

2 : FILE INCLUSION : A copying of one file to another files into program.

Ex : File inclusion of an external file "add.c".

```

#include<stdio.h>

#include add.c

void main()
{
    void add(); //FUNCTION PROTOTYPE/ DECLARATION.
    add();      //FUNCTION CALLING
}

```

The file **add1.c** contains the function definition as follows.

```

void add()
{
    int a,b,c;

    printf("enter two numbers");

    scanf("%d%d",&a,&b);

    c=a+b;

    printf("c value is:%d",c);
}

```

3 : CONDITIONAL COMPILATION DIRECTIVES : C preprocessor also supports number of conditional compilation directives as

- 1 : #undef : Undefined a macro
- 2 : #ifdef : Tests for a macro definition.
- 3 : #endif : Specifies the end of #if.
- 4 : #if : Tests compile-time condition.
- 5 : #else : Specifies alternative when #if test fails.

These are used to select a particular segment of code for compilation depending on the condition.

EXAMPLE PROGRAM FOR CONDITIONAL COMPILATION DIRECTIVES:

A) EXAMPLE PROGRAM FOR #IFDEF, #ELSE AND #ENDIF IN C:

- “#ifdef” directive checks whether particular macro is defined or not. If it is defined, “If” clause statements are included in source file.
- Otherwise, “else” clause statements are included in source file for compilation and execution.

Ex:

```
#include <stdio.h>

#define RAJU 100

int main()
{
    #ifdef RAJU

    printf("RAJU is defined. So, this line will be added in " \
        "this C file\n");

    #else

    printf("RAJU is not defined\n");

    #endif

    return 0;
}
```

OUTPUT:

RAJU is defined. So, this line will be added in this C file

B) EXAMPLE PROGRAM FOR #IFNDEF AND #ENDIF IN C:

- #ifndef exactly acts as reverse as #ifdef directive. If particular macro is not defined, “If” clause statements are included in source file.
- Otherwise, else clause statements are included in source file for compilation and execution.

Ex:

```
#include <stdio.h>
#define RAJU 100
int main()
{
    #ifndef SELVA
    {
        printf("SELVA is not defined. So, now we are going to " \
            "define here\n");
        #define SELVA 300
    }
    #else
    printf("SELVA is already defined in the program");

    #endif
    return 0;
}
```

OUTPUT:

SELVA is not defined. So, now we are going to define here

C) EXAMPLE PROGRAM FOR #IF, #ELSE AND #ENDIF IN C:

- “If” clause statement is included in source file if given condition is true.
- Otherwise, else clause statement is included in source file for compilation and execution.

Ex:

```
#include <stdio.h>
#define a 100
int main()
{
    #if (a==100)
    printf("This line will be added in this C file since " \
        "a \= 100\n");
    #else
    printf("This line will be added in this C file since " \
        "a is not equal to 100\n");
    #endif
    return 0;
}
```

OUTPUT:

This line will be added in this C file since a = 100

EXAMPLE PROGRAM FOR UNDEF IN C LANGUAGE:

This directive undefines existing macro in the program.

Ex:

```
#include <stdio.h>

#define height 100

void main()
{
    printf("First defined value for height  : %d\n",height);

    #undef height      // undefining variable

    #define height 600  // redefining the same for new value

    printf("value of height after undef \& redefine:%d",height);
}
```

OUTPUT:

First defined value for height : 100

value of height after undef & redefine : 600

EXAMPLE PROGRAM FOR PRAGMA IN C LANGUAGE:

Pragma is used to call a function before and after main function in a C program.

Ex:

```
#include <stdio.h>

void function1( );

void function2( );

#pragma startup function1

#pragma exit function2
```

```

int main( )
{
    printf ( "\n Now we are in main function" );
    return 0;
}

void function1( )
{
    printf("\nFunction1 is called before main function call");
}

void function2( )
{
    printf ( "\nFunction2 is called just before end of " \
            "main function" );
}

```

OUTPUT:

Function1 is called before main function call

Now we are in main function

Function2 is called just before end of main function

```

Ex :      #define TEST 1

          void main()
          {
              #ifdef TEST
              {
                  printf("This is compiled");

```

```

    }
    #else
    {
        printf("This is not compiled");
    }
    #endif
}

```

Ex2 : : #define FLAG 1

```

char ch;
void main()
{
    #if FLAG
    {
        ch='t';
        printf("This is compiled");
    }
    #else
    {
        ch='f';
        printf("This is not compiled");
    }
    #endif
}

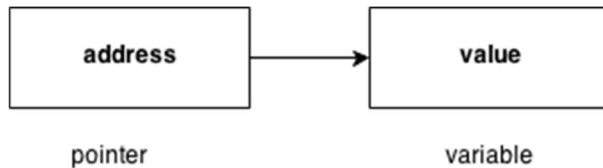
```

POINTERS:

Introduction

Definition:

Pointer is a variable that stores/hold address of another variable of same data type/ t is also known as locator or indicator that points to an address of a value. A pointer is a derived data type in C



Benefit of using pointers

- Pointers are more efficient in handling Array and Structure.
- Pointer allows references to function and thereby helps in passing of function as arguments to other function.
- It reduces length and the program execution time.
- It allows C to support dynamic memory management.

Declaration of Pointer

```
data_type* pointer_variable_name;
```

```
int* p;
```

Note: void type pointer works with all data types, but isn't used often.

Initialization of Pointer variable

Pointer Initialization is the process of assigning address of a variable to **pointer** variable. Pointer variable contains address of variable of same data type

```
int a = 10 ;
```

```
int *ptr ;    //pointer declaration
```

```
ptr = &a ;    //pointer initialization
```

or,

```
int *ptr = &a ; //initialization and declaration together
```

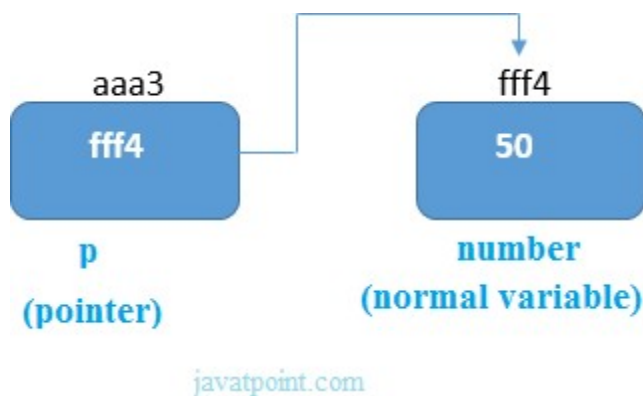
Note: Pointer variable always points to same type of data.

```
float a;
```

```
int *ptr;
```

```
ptr = &a; //ERROR, type mismatch
```

Above statement defines, p as pointer variable of type int. **Pointer example**



As you can see in the above figure, pointer variable stores the address of number variable i.e. fff4. The value of number variable is 50. But the address of pointer variable p is aaa3.

By the help of * (**indirection operator**), we can print the value of pointer variable p.

Reference operator (&) and Dereference operator (*)

& is called reference operator. It gives you the address of a variable. There is another operator that gets you the value from the address, it is called a dereference operator (*).

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	address of operator	determines the address of a variable.
* (asterisk sign)	indirection operator	accesses the value at the address.

Dereferencing of Pointer

Once a pointer has been assigned the address of a variable. To access the value of variable, pointer is dereferenced, using the **indirection operator** `*`.

```
int a,*p;  
  
a = 10;  
  
p = &a;  
  
printf("%d",*p); //this will print the value of a.  
  
printf("%d",&a); //this will also print the value of a.  
  
printf("%u",&a); //this will print the address of a.  
  
printf("%u",p); //this will also print the address of a.  
  
printf("%u",&p); //this will also print the address of p.
```

KEY POINTS TO REMEMBER ABOUT POINTERS IN C:

- Normal variable stores the value whereas pointer variable stores the address of the variable.
- The content of the C pointer always be a whole number i.e. address.
- Always C pointer is initialized to null, i.e. `int *p = null`.
- The value of null pointer is 0.
- `&` symbol is used to get the address of the variable.
- `*` symbol is used to get the value of the variable that the pointer is pointing to.
- If a pointer in C is assigned to NULL, it means it is pointing to nothing.
- Two pointers can be subtracted to know how many elements are available between these two pointers.
- But, Pointer addition, multiplication, division are not allowed.
- The size of any pointer is 2 byte (for 16 bit compiler).

Example:

```
#include <stdio.h>

#include <conio.h>

void main(){

int number=50;

int *p;

clrscr();

p=&number;//stores the address of number variable

printf("Address of number variable is %x \n",&number);

printf("Address of p variable is %x \n",p);

printf("Value of p variable is %d \n",*p);

getch();

}
```

Output

Address of number variable is fff4

Address of p variable is fff4

Value of p variable is 50

Example:

```
#include <stdio.h>

int main()

{

int *ptr, q;

q = 50;
```



```

/* address of q is assigned to ptr */

ptr = &q;

/* display q's value using ptr variable */

printf("%d", *ptr);

return 0;

}

```

Output

50

Example:

```

#include <stdio.h>

int main()
{
    int var=10;

    int *p;

    p= &var;

    printf ( "\n Address of var is: %u", &var);

    printf ( "\n Address of var is: %u", p);

    printf ( "\n Address of pointer p is: %u", &p);

    /* Note I have used %u for p's value as it should be an address*/

    printf( "\n Value of pointer p is: %u", p);

    printf ( "\n Value of var is: %d", var);

    printf ( "\n Value of var is: %d", *p);

    printf ( "\n Value of var is: %d", *( &var));

}

```

Output:

Address of var is: 00XBBA77

Address of var is: 00XBBA77

Address of pointer p is: 77221111

Value of pointer p is: 00XBBA77

Value of var is: 10

Value of var is: 10

Value of var is: 10

NULL Pointer

A pointer that is not assigned any value but NULL is known as NULL pointer. If you don't have any address to be specified in the pointer at the time of declaration, you can assign NULL value.

Or

It is always a good practice to assign a NULL value to a pointer variable in case you do not have an exact address to be assigned. This is done at the time of variable declaration. A pointer that is assigned NULL is called a **null** pointer. `int *p=NULL;`

Note: The NULL pointer is a constant with a value of zero defined in several standard libraries/ in most the libraries, the value of pointer is 0 (zero)

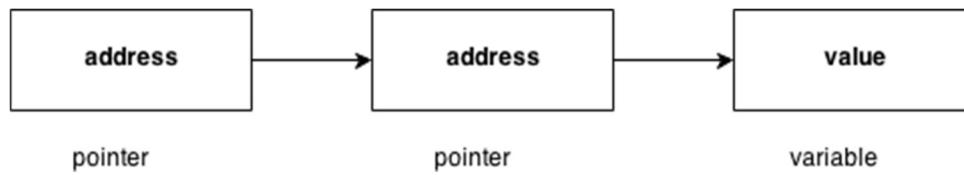
Example:

The value of ptr is 0

Pointers for Inter-Function Communication**Pointers to Pointers**

Pointers can point to other pointers /pointer refers to the address of another pointer.

pointer can point to the address of another pointer which points to the address of a value.

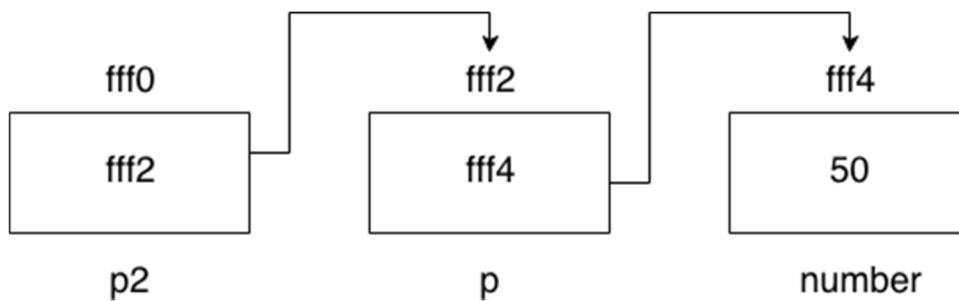


syntax of pointer to pointer

```
int **p2;
```

pointer to pointer example

Let's see an example where one pointer points to the address of another pointer.



Example:

```
#include <stdio.h>

#include <conio.h>

void main(){

int number=50;

int *p;//pointer to int

int **p2;//pointer to pointer

clrscr();

p=&number;//stores the address of number variable

p2=&p;

printf("Address of number variable is %x \n",&number);

printf("Address of p variable is %x \n",p);
```

```

printf("Value of *p variable is %d \n",*p);
printf("Address of p2 variable is %x \n",p2);
printf("Value of **p2 variable is %d \n",**p);
getch();
}

```

Output

Address of number variable is fff4

Address of p variable is fff4

Value of *p variable is 50

Address of p2 variable is fff2

Value of **p variable is 50

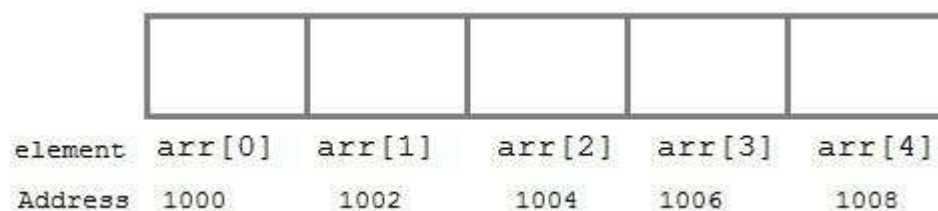
Arrays and Pointers

When an array is declared, compiler allocates sufficient amount of memory to contain all the elements of the array. Base address which gives location of the first element is also allocated by the compiler.

Suppose we declare an array arr,

```
int arr[5]={ 1, 2, 3, 4, 5 };
```

Assuming that the base address of **arr** is 1000 and each integer requires two byte, the five element will be stored as follows



Here variable **arr** will give the base address, which is a constant pointer pointing to the element, **arr[0]**. Therefore **arr** is containing the address of **arr[0]** i.e 1000.

arr is equal to &arr[0] // by default

We can declare a pointer of type int to point to the array arr.

```
int arr[5]={ 1, 2, 3, 4, 5 };
```

```
int *p;
```

```
p = arr;
```

```
or p = &arr[0]; //both the statements are equivalent.
```

Now we can access every element of array **arr** using **p++** to move from one element to another.

NOTE : You cannot decrement a pointer once incremented. **p--** won't work.

Pointer to Array

we can use a pointer to point to an Array, and then we can use that pointer to access the array. Lets have an example,

```
int i;
```

```
int a[5] = {1, 2, 3, 4, 5};
```

```
int *p = a; // same as int*p = &a[0]
```

```
for (i=0; i<5; i++)
```

```
{
```

```
    printf("%d", *p);
```

```
    p++;
```

```
}
```

In the above program, the pointer ***p** will print all the values stored in the array one by one. We can also use the Base address (**a** in above case) to act as pointer and print all the values.

Replacing the **printf("%d", *p);** statement of above example, with below mentioned statements. Lets see what will be the result.

printf("%d", a[i]); → **prints the array, by incrementing index**

printf("%d", i[a]); → **this will also print elements of array**

printf("%d", a+i); → **This will print address of all the array elements**

printf("%d", *(a+i)); → **Will print value of array element.**

printf("%d", *a); → **will print value of a[0] only**

a++; → **Compile time error, we cannot change base address of the array.**

Relation between Arrays and Pointers

Consider an array:

```
int arr[4];
```

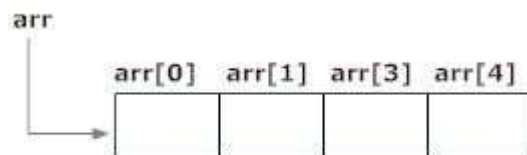


Figure: Array as Pointer

In C programming, name of the array always points to address of the first element of an array.

In the above example, **arr** and **&arr[0]** points to the address of the first element.

&arr[0] is equivalent to arr

Since, the addresses of both are the same, the values of **arr** and **&arr[0]** are also the same.

arr[0] is equivalent to *arr (value of an address of the pointer)

Similarly,

`&arr[1]` is equivalent to `(arr + 1)` AND, `arr[1]` is equivalent to `*(arr + 1)`.

`&arr[2]` is equivalent to `(arr + 2)` AND, `arr[2]` is equivalent to `*(arr + 2)`.

`&arr[3]` is equivalent to `(arr + 3)` AND, `arr[3]` is equivalent to `*(arr + 3)`.

.

.

`&arr[i]` is equivalent to `(arr + i)` AND, `arr[i]` is equivalent to `*(arr + i)`.

Example: Program to find the sum of six numbers with arrays and pointers

```
#include <stdio.h>

int main()
{
    int i, classes[6], sum = 0;
    printf("Enter 6 numbers:\n");
    for(i = 0; i < 6; ++i)
    {
        // (classes + i) is equivalent to &classes[i]
        scanf("%d", (classes + i));
        // *(classes + i) is equivalent to classes[i]
        sum += *(classes + i);
    }
    printf("Sum = %d", sum);
    return 0;
}
```

Output

Enter 6 numbers:

2

3

4

5

3

4

Sum = 21

Pointer Arithmetic and Arrays

pointer holds address of a value, so there can be arithmetic operations on the pointer variable.

There are four arithmetic operators that can be used on pointers:

- Increment(++)
- Decrement(--)
- Addition(+)
- Subtraction(-)

Increment pointer:

1. Incrementing Pointer is generally used in array because we have contiguous memory in array and we know the contents of next memory location.
2. Incrementing Pointer Variable Depends Upon data type of the Pointer variable.

The formula of incrementing pointer is given below:

$\text{new_address} = \text{current_address} + i * \text{size_of}(\text{data type})$

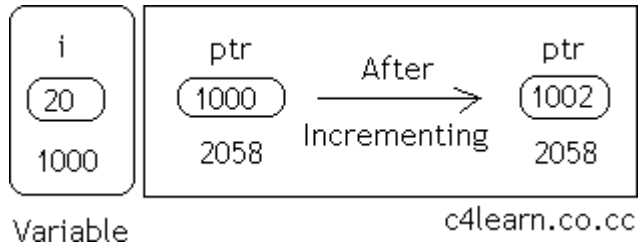
Three rules should be used to increment pointer

$\text{Address} + 1 = \text{Address}$

$\text{Address}++ = \text{Address}$

`++Address = Address`

Pictorial Representation :



Data Type	Older Address stored in pointer	Next Address stored in pointer after incrementing (ptr++)
int	1000	1002
float	1000	1004
char	1000	1001

Note :

32 bit

For 32 bit int variable, it will increment to 2 byte.

64 bit

For 64 bit int variable, it will increment to 4 byte.

Example:

```
#include <stdio.h>
```

```
void main(){
```

```
int number=50;
```

```

int *p;//pointer to int

p=&number;//stores the address of number variable

printf("Address of p variable is %u \n",p);

p=p+1;

printf("After increment: Address of p variable is %u \n",p);

}

```

Output

Address of p variable is 3214864300

After increment: Address of p variable is 3214864304

Decrement(--)

Like increment, we can decrement a pointer variable.

formula of decrementing pointer

$$\text{new_address} = \text{current_address} - i * \text{size_of}(\text{data type})$$

Example:

```

#include <stdio.h>

void main(){

    int number=50;

    int *p;//pointer to int

    p=&number;//stores the address of number variable

    printf("Address of p variable is %u \n",p);

    p=p-1;
}

```

```
printf("After decrement: Address of p variable is %u \n",p);  
  
}
```

Output

Address of p variable is 3214864300

After decrement: Address of p variable is 3214864296

Addition(+)

We can add a value to the pointer variable.

formula of adding value to pointer

$$\text{new_address} = \text{current_address} + (\text{number} * \text{size_of}(\text{data type}))$$

Note:

32 bit

For 32 bit int variable, it will add 2 * number.

64 bit

For 64 bit int variable, it will add 4 * number.

Example:

```
#include <stdio.h>  
  
void main(){  
  
    int number=50;  
  
    int *p;//pointer to int  
  
    p=&number;//stores the address of number variable  
  
    printf("Address of p variable is %u \n",p);
```

```
p=p+3; //adding 3 to pointer variable

printf("After adding 3: Address of p variable is %u \n",p);

}
```

Output

Address of p variable is 3214864300

After adding 3: Address of p variable is 3214864312

Subtraction (-)

Like pointer addition, we can subtract a value from the pointer variable. The formula of subtracting value from pointer variable.

$$\text{new_address} = \text{current_address} - (\text{number} * \text{size_of}(\text{data type}))$$

Example:

```
#include <stdio.h>

void main(){

int number=50;

int *p;//pointer to int

p=&number;//stores the address of number variable

printf("Address of p variable is %u \n",p);

p=p-3; //subtracting 3 from pointer variable

printf("After subtracting 3: Address of p variable is %u \n",p);

}
```

Output

Address of p variable is 3214864300

After subtracting 3: Address of p variable is 3214864288

Passing an Array to a Function

If you want to pass a single-dimension array as an argument in a function, you would have to declare a formal parameter in one of following three ways and all three declaration methods produce similar results because each tells the compiler that an integer pointer is going to be received. Similarly, you can pass multi-dimensional arrays as formal parameters.

1) Formal parameters as a pointer –

```
void myFunction(int *param) {  
    .  
    .  
    .  
}
```

2) Formal parameters as a sized array –

```
void myFunction(int param[10]) {  
    .  
    .  
    .  
}
```

3) Formal parameters as an unsized array –

```
void myFunction(int param[10]) {  
    .  
    .  
    .  
}
```

Example1: pass an entire array to a function argument

```
#include <stdio.h>  
/* function declaration */  
double getAverage(int arr[], int size);  
int main () {  
    /* an int array with 5 elements */  
    int balance[5] = {1000, 2, 3, 17, 50};
```

```

double avg;

/* pass pointer to the array as an argument */
avg = getAverage( balance, 5 );
/* output the returned value */
printf( "Average value is: %f", avg );
return 0;
}
double getAverage(int arr[], int size) {

int i;
double avg;
double sum = 0;

for (i = 0; i < size; ++i) {
    sum += arr[i];
}

avg = sum / size;

return avg;
}

```

Output

Average value is: 214.400000

Example2: pass an entire array to a function argument

```

#include <stdio.h>
myfuncn( int *var1, int var2)
{
    for(int x=0; x<var2; x++)
    {
        printf("Value of var_arr[%d] is: %d \n", x, *var1);
        /*increment pointer for next element fetch*/
        var1++;
    }
}

```

```

int main()
{
    int var_arr[] = {11, 22, 33, 44, 55, 66, 77};
    myfuncn(&var_arr, 7);
    return 0;
}

```

Output

Value of var_arr[0] is: 11
 Value of var_arr[1] is: 22
 Value of var_arr[2] is: 33
 Value of var_arr[3] is: 44
 Value of var_arr[4] is: 55
 Value of var_arr[5] is: 66
 Value of var_arr[6] is: 77

Example: Call by value method –

```

#include <stdio.h>

disp( char ch)
{
    printf("%c ", ch);
}

int main()
{
    char arr[] = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'};
    for (int x=0; x<=10; x++)
    {
        /* I'm passing each element one by one using subscript*/
        disp (arr[x]);
    }

    return 0;
}

```

Output:

a b c d e f g h i j

In this method of calling a function, the actual arguments gets copied into formal arguments. In this example actual argument(or parameter) is arr[x] and formal parameter is ch.

Example: Call by reference method: Using pointers

```
#include <stdio.h>

disp( int *num)

{

    printf("%d ", *num);

}

int main()

{

    int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 0};

    for (int i=0; i<=10; i++)

    {

        /* I'm passing element's address*/

        disp (&arr[i]);

    }

    return 0;

}
```

Output:

1 2 3 4 5 6 7 8 9 0

Array of Pointers

An array of pointers would be an array that holds memory locations. An array of pointers is an indexed set of [variables](#) in which the variables are [pointers](#) (a reference to a location in [memory](#)).

Syntax:

```
data_type_name * variable name
```

Example

```
int *ptr[MAX];
```

Array alpha []	Pointer <i>a</i>
alpha[0]	*a
alpha[1]	*(a+1)
alpha[2]	*(a+2)
alpha[3]	*(a+3)
alpha[n]	*(a+n)

Example1:

```
#include <stdio.h>
```

```
const int MAX = 3;
```

```
int main () {
```

```
    int var[] = {10, 100, 200};
```

```
    int i, *ptr[MAX];
```

```
    for ( i = 0; i < MAX; i++) {
```

```

    ptr[i] = &var[i]; /* assign the address of integer. */

}

for ( i = 0; i < MAX; i++) {

    printf("Value of var[%d] = %d\n", i, *ptr[i] );

}

return 0;

}

```

Output

Value of var[0] = 10

Value of var[1] = 100

Value of var[2] = 200

Example2:

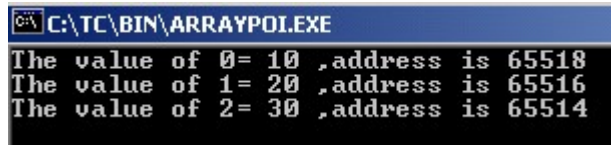
```

#include <stdio.h>
#include <conio.h>
main() {
    clrscr();
    int *array[3];
    int x = 10, y = 20, z = 30;
    int i;
    array[0] = &x;
    array[1] = &y;
    array[2] = &z;
    for (i=0; i< 3; i++) {
        printf("The value of %d= %d ,address is %u\t \n", i, *(array[i]),
        array[i]);
    }
    getch();
}

```

```
return 0;
}
```

Output



```
C:\TC\BIN\ARRAYPOLEXE
The value of 0= 10 ,address is 65518
The value of 1= 20 ,address is 65516
The value of 2= 30 ,address is 65514
```

Example3:

```
#include <stdio.h>

const int MAX = 4;

int main () {

    char *names[] = {

        "Zara Ali",

        "Hina Ali",

        "Nuha Ali",

        "Sara Ali"

    };

    int i = 0;

    for ( i = 0; i < MAX; i++) {

        printf("Value of names[%d] = %s\n", i, names[i] );

    }

    return 0;

}
```

Output:

Value of names[0] = Zara Ali

Value of names[1] = Hina Ali

Value of names[2] = Nuha Ali

Value of names[3] = Sara Ali

Example4:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
char *fruit[] = {
```

```
"watermelon",
```

```
"banana",
```

```
"pear",
```

```
"apple",
```

```
"coconut",
```

```
"grape",
```

```
"blueberry"
```

```
};
```

```
int x;
```

```
for(x=0;x<7;x++)
```

```
puts(fruit[x]);
```

```
return(0);
```

```
}
```

Pointers to Void and to Functions

Pointers to Void

Note:

1. Suppose we have to declare integer pointer, character pointer and float pointer then we need to declare 3 pointer variables.
2. Instead of declaring [different types of pointer variable](#) it is feasible to declare single pointer variable which can act as integer pointer, character pointer.

A pointer variable declared using a particular data type can not hold the location address of variables of other data types. It is invalid and will result in a compilation error.

Ex:- `char *ptr;`

```
int var1;
```

```
ptr=&var1; // This is invalid because „ptr“ is a character pointer variable.
```

Here comes the importance of a “void pointer”. A void pointer is nothing but a pointer variable declared using the reserved word in C „void”.

Void Pointer Basics :

3. In C **General Purpose Pointer** is called as void Pointer.
4. It does not have any data type associated with it
5. It can store address of any type of variable
6. A void pointer is a C convention for a raw address.
7. The compiler has no idea what type of object a void Pointer really points to ?

Void pointer: A void pointer is a pointer that has no associated data type with it. A void pointer can hold address of any type and can be typecasted to any type. **Special type of pointer** called void pointer or **general purpose pointer**.

Declaration of void pointer

```
void * pointer_name;
```

Void pointer example

```
void *ptr; // ptr is declared as Void pointer
```

```
char cnum;
```

```
int inum;
```

```
float fnum;
```

```
ptr = &cnum; // ptr has address of character data
```

```
ptr = &inum; // ptr has address of integer data
```

```
ptr = &fnum; // ptr has address of float data
```

Advantages of void pointers:

1) malloc() and calloc() return void * type and this allows these functions to be used to allocate memory of any data type (just because of void *)

```
int main(void)
```

```
{
```

```
    // Note that malloc() returns void * which can be
```

```
    // typecasted to any type like int *, char *, ..
```

```
int *x = malloc(sizeof(int) * n);  
  
}
```

2) void pointers in C are used to implement generic functions in C.

Note:

1) void pointers cannot be dereferenced. For example the following program doesn't compile.

```
#include<stdio.h>  
  
int main()  
{  
  
    int a = 10;  
  
    void *ptr = &a;  
  
    printf("%d", *ptr);  
  
    return 0;  
  
}
```

Output:

Compiler Error: 'void*' is not a pointer-to-object type

The following program compiles and runs fine.

```
#include<stdio.h>  
  
int main()  
{  
  
    int a = 10;  
  
    void *ptr = &a;  
  
    printf("%d", *(int *)ptr);  
  
    return 0;  
  
}
```

Output:

10

Summary : Void Pointer

Scenario	Behavior
When We assign address of integer variable to void pointer	Void Pointer Becomes Integer Pointer
When We assign address of character variable to void pointer	Void Pointer Becomes Character Pointer
When We assign address of floating variable to void pointer	Void Pointer Becomes Floating Pointer

Pointers to functions/ Function Pointers

- A pointer to a function points to the address of the executable code of the function.
- We can use pointers to call functions and to pass functions as arguments to other functions.
- We cannot perform pointer arithmetic on pointers to functions.
- The type of a pointer to a function is based on both the return type and parameter types of the function.
- A declaration of a pointer to a function must have the pointer name in parentheses.
- The function call operator () has a higher precedence than the dereference operator *. Without them, the compiler interprets the statement as a function that returns a pointer to a specified return type.

declare Pointer to function?

<function return type>(*<Pointer_name>)(function argument list)

For example –

For example:

1) `int *f(int a); /* function f returning an int * */`

In this declaration, f is interpreted as a function that takes an int as argument, and returns a pointer to an int.

2) double (*p2f)(double, char)

Here double is a return type of function, p2f is pointer name & (double, char) is an argument list for the function. Which means the first argument for this function should be double and the second one would be of char type.

Example:

```
#include<stdio.h>

int sum (int num1, int num2)
{
    return sum1+sum2;
}

int main()
{
    int (*f2p) (int, int);
    f2p = sum;
    int op1 = f2p (10, 13);
    int op2 = sum (10, 13);
    printf("Output 1 – for function call via Pointer: %d",op1);
    printf("Output2 – for direct function call: %d", op2);
    return 0;
}
```

Output:

Output 1 – for function call via Pointer: 23

Output2 – for direct function call: 23

You would have noticed that the output of both the statements is same –
f2p(10, 13) == sum(10, 13)

which means in generic sense you can write it out as:

pointer_name(argument list) == function(same argument list)

wherein pointer_name is declared as:
return_type(*pointer_name)(argument list);
pointer_name = function_name(argument list);

Memory Allocation Functions

The concept of **dynamic memory allocation in c language** enables the C programmer to allocate memory at runtime.

Or

The process of allocating memory at runtime is known as **dynamic memory allocation**. Library routines known as "memory management functions" are used for allocating and freeing memory during execution of a program. These functions are defined in **stdlib.h**.

Dynamic memory allocation in c language is possible by 4 functions of stdlib.h header file.

1. malloc()
2. calloc()
3. realloc()
4. free()

Difference between static memory allocation and dynamic memory allocation.

static memory allocation	dynamic memory allocation
memory is allocated at compile time.	memory is allocated at run time.
memory can't be increased while executing program.	memory can be increased while executing program.
used in array.	used in linked list.

Methods used for dynamic memory allocation.

malloc()	allocates single block of requested memory.
calloc()	allocates multiple block of requested memory.
realloc()	reallocates the memory occupied by malloc() or calloc() functions.
free()	frees the dynamically allocated memory.

Note: Dynamic memory allocation related function can be applied for any data type that's why dynamic memory allocation related functions return void*.

Memory Allocation Process

Global variables, **static** variables and program instructions get their memory in **permanent** storage area whereas **local** variables are stored in area called **Stack**. The memory space between these two region is known as **Heap** area. This region is used for dynamic memory allocation during execution of the program. The size of heap keep changing.

malloc()

malloc stands for "**memory allocation**".

The malloc() function allocates single block of requested memory at runtime. This function reserves a block of memory of given size and returns a pointer of type void. This means that we can assign it to any type of pointer using typecasting. It doesn't initialize memory at execution time, so it has garbage value initially. If it fails to locate enough space (memory) it returns a NULL pointer.

syntax

```
ptr=(cast-type*)malloc(byte-size)
```

Example

```
int *x;
```

```
x = (int*)malloc(100 * sizeof(int)); //memory space allocated to variable x
```

```
free(x); //releases the memory allocated to variable x
```

This statement will allocate either 200 or 400 according to size of int 2 or 4 bytes respectively and the pointer points to the address of first byte of memory.

Example

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num, i, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &num);
    ptr = (int*) malloc(num * sizeof(int)); //memory allocated using malloc
    if(ptr == NULL)
    {
        printf("Error! memory not allocated.");
        exit(0);
    }
    printf("Enter elements of array: ");
    for(i = 0; i < num; ++i)
    {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
    free(ptr);
    return 0;
}
```

calloc()

calloc stands for "**contiguous allocation**".

Calloc() is another memory allocation function that is used for allocating memory at runtime. **calloc** function is normally used for allocating memory to derived data types such as **arrays** and **structures**. The calloc() function allocates multiple block of requested memory.

It initially initialize (sets) all bytes to zero. If it fails to locate enough space(memory) it returns a NULL pointer. The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size.

Syntax

```
ptr = (cast-type*)calloc(n/number, element-size);
```

calloc() required 2 arguments of type count, size-type.

Count will provide number of elements; size-type is data type size

Example

```
int*arr;  
arr=(int*)calloc(10, sizeof(int));    // 20 byte  
char*str;  
str=(char*)calloc(50, sizeof(char)); // 50 byte
```

Example

```
struct employee  
{  
    char *name;  
    int salary;  
};  
typedef struct employee emp;  
emp *e1;  
e1 = (emp*)calloc(30, sizeof(emp));
```

Example

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num, i, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &num);
    ptr = (int*) calloc(num, sizeof(int));
    if(ptr == NULL)
    {
        printf("Error! memory not allocated.");
        exit(0);
    }
    printf("Enter elements of array: ");
    for(i = 0; i < num; ++i)
    {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
    free(ptr);
    return 0;
}
```

Difference between malloc() and calloc()

calloc()	malloc()
calloc() initializes the allocated memory with 0 value.	malloc() initializes the allocated memory with garbage values.
Number of arguments is 2	Number of argument is 1
Syntax : (cast_type *)calloc(blocks , size_of_block);	Syntax : (cast_type *)malloc(Size_in_bytes);

realloc(): changes memory size that is already allocated to a variable.

Or

If the previously allocated memory is insufficient or more than required, you can change the previously allocated memory size using realloc().

- If memory is not sufficient for malloc() or calloc(), you can reallocate the memory by realloc() function. In short, it changes the memory size. By using realloc() we can create the memory dynamically at middle stage. Generally by using realloc() we can reallocation the memory. Realloc() required 2 arguments of type void*, size_type. Void* will indicates previous block base address, size-type is data type size. Realloc() will creates the memory in bytes format and initial value is garbage.

syntax

```
ptr=realloc(ptr, new-size)
```

Example

```
int *x;
```

```
x=(int*)malloc(50 * sizeof(int));
```

```
x=(int*)realloc(x,100); //allocated a new memory to variable x
```

Example

```
void*realloc(void*, size-type);  
int *arr;  
arr=(int*)calloc(5, sizeof(int));  
.....  
.....  
....  
arr=(int*)realloc(arr,sizeof(int)*10);
```

Example:

```
#include <stdio.h>  
#include <stdlib.h>  
int main()  
{  
    int *ptr, i , n1, n2;  
    printf("Enter size of array: ");  
    scanf("%d", &n1);  
    ptr = (int*) malloc(n1 * sizeof(int));  
    printf("Address of previously allocated memory: ");  
    for(i = 0; i < n1; ++i)  
        printf("%u\t",ptr + i);  
    printf("\nEnter new size of array: ");  
    scanf("%d", &n2);  
    ptr = realloc(ptr, n2);  
    for(i = 0; i < n2; ++i)  
        printf("%u\t", ptr + i);  
    return 0;  
}
```


free()

When your program comes out, operating system automatically release all the memory allocated by your program but as a good practice when you are not in need of memory anymore then you should release that memory by calling the function free().

The memory occupied by malloc() or calloc() functions must be released by calling free() function. Otherwise, it will consume memory until program exit.

Or

Dynamically allocated memory created with either calloc() or malloc() doesn't get freed on its own. You must explicitly use free() to release the space.

Syntax:

```
free(ptr);
```

Example

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int num, i, *ptr, sum = 0;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &num);
```

```
    ptr = (int*) malloc(num * sizeof(int)); //memory allocated using malloc
```

```
    if(ptr == NULL)
```

```
    {
```

```
        printf("Error! memory not allocated.");
```

```
        exit(0);
```

```
    }
```

```
    printf("Enter elements of array: ");
```

```
    for(i = 0; i < num; ++i)
```

```
    {
```

```
        scanf("%d", ptr + i);
```

```

        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
    free(ptr);
    return 0;
}

```

Command-Line Arguments:

It is possible to pass some values from the command line to your C programs when they are executed. These values are called **command line arguments** and many times they are important for your program especially when you want to control your program from outside instead of hard coding those values inside the code.

The arguments passed from command line are called command line arguments. These arguments are handled by main() function.

To support command line argument, you need to change the structure of main() function

Syntax:

```
int main(int argc, char *argv[] )
```

Here, **argc** counts the number of arguments. It counts the file name as the first argument.

The **argv[]** contains the total number of arguments. The first argument is the file name always.

Example1

```

#include <stdio.h>

int main( int argc, char *argv[] ) {
    if( argc == 2 ) {
        printf("The argument supplied is %s\n", argv[1]);
    }
    else if( argc > 2 ) {
        printf("Too many arguments supplied.\n");
    }
    else {
        printf("One argument expected.\n");
    }
}

```

```
}
```

Output

Example2

```
#include <stdio.h>

void main(int argc, char *argv[] ) {
    printf("Program name is: %s\n", argv[0]);
    if(argc < 2){
        printf("No argument passed through command line.\n");
    }
    else{
        printf("First argument is: %s\n", argv[1]);
    }
}
```

Output

```
program.exe hello
Program name is: program
First argument is: hello
```

Note

But if you pass many arguments within double quote, all arguments will be treated as a single argument only.

Example

```
./program "hello c how r u"
Program name is: program
First argument is: hello c how r u
```

You can write your program to print all the arguments. In this program, we are printing only argv[1], that is why it is printing only one argument.

Example3

```
#include<stdio.h>
#include<conio.h>
void main(int argc, char* argv[])
{
    int i;
    clrscr();
    printf("Total number of arguments: %d",argc);
    for(i=0;i< argc;i++)
    {
        printf("\n %d argument: %s",i,argv[i]);
        getch();
    }
}
```

Output

C:/TC/BIN>TCC mycmd.c

C:/TC/BIN>mycmd 10 20

Number of Arguments: 3

0 arguments c:/tc/bin/mycmd.exe

1 arguments: 10

2 arguments: 20

Note: In above output we passed two arguments but it shows "Number of Arguments: 3" because **argc** takes Number of arguments in the command line including program name. So here two arguments and one program name (mycmd.exe) total 3 arguments.

```
C:\Windows\system32\cmd.exe - tcc mycmd.c

C:\TC\BIN>tcc mycmd.c ----- Compile program

Turbo C++ Version 3.00 Copyright (c) 1992 Borland International
mycmd.c:
Turbo Link Version 5.0 Copyright (c) 1992 Borland International

Available memory 4108672

C:\TC\BIN>mycmd 10 20 ----- Run program with two Arguments

Total number of arguments: 3
0 argument: C:\TC\BIN\MYCMD.EXE ----- 1 st argement is program name
1 argument: 10 ----- Arguments
2 argument: 20 ----- Arguments
C:\TC\BIN>

Note: here show 3 arguments because argc take Number of
arguments in the command line including program name.

Tutorial4us.com
```

Example4:

```
#include<stdio.h>
#include<conio.h>
void main(int argc, char* argv[])
{
clrscr();

printf("\n Program name : %s \n", argv[0]);
printf("1st arg : %s \n", argv[1]);
printf("2nd arg : %s \n", argv[2]);
printf("3rd arg : %s \n", argv[3]);
printf("4th arg : %s \n", argv[4]);
printf("5th arg : %s \n", argv[5]);
getch();
}
```

Output

```
C:/TC/BIN>TCC mycmd.c
C:/TC/BIN>mycmd this is a program
```

Program name : c:/tc/bin/mycmd.c

1st arg : this

2nd arg : is

3rd arg : a

4th arg : program

5th arg : (null)

Explanation: In the above example.

```
argc    = 5
```

```
argv[0] = "mycmd"
```

```
argv[1] = "this"
```

```
argv[2] = "is"
```

```
argv[3] = "a"
```

```
argv[4] = "program"
```

```
argv[5] = NULL
```

Why command line arguments program not directly run from TC IDE

Command line arguments related programs are not execute directly from TC IDE because arguments can not be passed.

Edit Command Line Argument Program

To Edit the Command Line Argument Program use **edit** Command.

Syntax

```
C:/cprogram>edit mycmd.c
```

UNIT IV

STRUCTURES, UNIONS, ENUMERATIONS AND TYPEDEF

Structure Definition

Structure is a user defined data type which hold or store heterogeneous/different types data item or element in a single variable. It is a Combination of primitive and derived data type.

or

A structure is a collection of one or more data items of different data types, grouped together under a single name.

Variables inside the structure are called members of structure.

Each element of a structure is called a member.

struct keyword is used to define/create a structure. **struct** define a new data type which is a collection of different type of data.

Syntax

```
struct structure_name /tag name
```

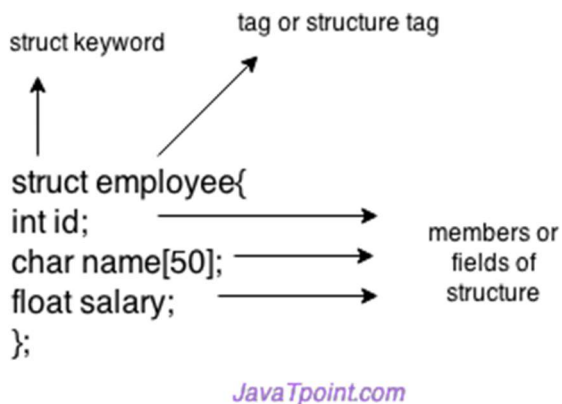
```
{  
    data_type member1;  
    data_type member2;  
    .  
    .  
    data_type member n;  
};
```

Note: Don't forget the semicolon }; in the ending line.

Example

```
struct employee  
{ int id;  
  char name[50];  
  float salary;  
};
```

Here, **struct** is the keyword, **employee** is the tag name of structure; **id**, **name** and **salary** are the members or fields of the structure. Let's understand it by the diagram given below:



Syntax to create structure variable

```
struct tagname/structure_name variable;
```

Declaring structure variable

We can declare variable for the structure, so that we can access the member of structure easily. There are two ways to declare structure variable:

1. By struct keyword within main() function/ Declaring Structure variables separately

2. By declaring variable at the time of defining structure/ Declaring Structure Variables with Structure definition

1st way:

Let's see the example to declare structure variable by struct keyword. It should be declared within the main function.

```
struct employee  
  
{ int id;  
  char name[50];  
  
  float salary;  
  
};
```

Now write given code inside the main() function.

```
struct employee e1, e2;
```

2nd way:

Let's see another way to declare variable at the time of defining structure.

```
struct employee  
  
{ int id;  
  char name[50];  
  
  float salary;  
  
}e1,e2;
```

Which approach is good

But if no. of variable are not fixed, use 1st approach. It provides you flexibility to declare the structure variable many times.

If no. of variables are fixed, use 2nd approach. It saves your code to declare variable in main() function.

Structure Initialization

structure variable can also be initialized at compile time.

```
struct Patient
```

```
{
```

```
float height;
```

```
int weight;
```

```
int age;
```

```
};
```

```
struct Patient p1 = { 180.75 , 73, 23 }; //initialization
```

or

```
struct patient p1;
```

```
p1.height = 180.75; //initialization of each member separately
```

```
p1.weight = 73;
```

```
p1.age = 23;
```

Accessing Structures/ Accessing members of structure

There are two ways to access structure members:

1. By . (member or dot operator)
2. By -> (structure pointer operator)

When the variable is normal type then go for struct to member operator.

When the variable is pointer type then go for pointer to member operator.

Any member of a structure can be accessed as:

```
structure_variable_name.member_name
```

Example

```
struct book
{
char name[20];
char author[20];
int pages;
};
```

```
struct book b1;
```

for accessing the structure members from the above example

b1.name, b1.author, b1.pages:

Example

```
struct emp
{
int id;
char name[36];
int sal;
};

sizeof(struct emp)    // --> 40 byte (2byte+36byte+2byte)
```

Example of Structure in C

```
#include<stdio.h>
#include<conio.h>

struct emp
{
int id;
char name[36];
float sal;
```

```

};

void main()

{
    struct emp e;

    clrscr();

    printf("Enter employee Id, Name, Salary: ");
    scanf("%d",&e.id);

    scanf("%s",&e.name);

    scanf("%f",&e.sal);

    printf("Id: %d",e.id);

    printf("\nName: %s",e.name);

    printf("\nSalary: %f",e.sal);

    getch();
}

```

Output

Output: Enter employee Id, Name, Salary: 5 Spidy 45000

Id : 05

Name: Spidy

Salary: 45000.00

Example

```

#include <stdio.h>

#include <string.h>

struct employee

{   int id;

    char name[50];

```

```
}e1; //declaring e1 variable for structure  
  
int main( )  
{  
    //store first employee information
```